МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ ПП ПМ.04

Сопровождение настольного журнала. Диагностика, обмен CSV, обновление и откат.

Сервисный журнал учебного центра — демонстрационный образец. Назначенная область и её данные сохраняются.

СОДЕРЖАНИЕ

1.1. Сопровождение существующего журнала 4

1.2. Обязательные функции и ограничения 5

1.3. Диагностика и самостоятельная работа 6

1.4. Дополнительный результат длинной практики 7

1.5. Приёмка обновления 7

1.6. Сдача результата 8

2. Файлы для выполнения и сверки 10

3.1. От исходного приложения к проверенному обновлению 12

3.2. Открытие и контроль исходной версии 13

3.3. Контрольная дата как входное значение 15

3.4. Подключение CSV без потери исходной проверки 16

3.5. Входные данные и воспроизведение трёх дефектов 17

3.6. D1: результат чтения содержит данные и предупреждения 19

3.7. D2: запись соблюдает тот же контракт, что чтение 25

3.8. D3: просрочка зависит от срока и состояния 26

3.9. Отбор для оператора без потери исходного списка 27

3.10. Как перенести правило на индивидуальную область 31

3.11. Регрессионная матрица и диагностика ошибок 32

3.12. Release, резервная копия и передача обновления 35

3.13. Проверенный откат и работа по своему варианту 36

3.14. Расширенный маршрут: столбцы по именам 38

4. Полные файлы для сверки 42

4.1. Исходная версия: полный файл ServiceDesk.csproj для сверки 42

4.2. Исходная версия: полный файл App.xaml для сверки 42

4.3. Исходная версия: полный файл App.xaml.cs для сверки 43

4.4. Исходная версия: полный файл MainWindow.xaml для сверки 45

4.5. Исходная версия: полный файл MainWindow.xaml.cs для сверки 47

4.6. Исправленная версия: полный файл ServiceDesk-fixed.csproj для сверки 48

4.7. Исправленная версия: полный файл App.xaml для сверки 49

4.8. Исправленная версия: полный файл App.xaml.cs для сверки 50

4.9. Исправленная версия: полный файл MainWindow.xaml для сверки 52

4.10. Исправленная версия: полный файл MainWindow.xaml.cs для сверки 54

4.11. Расширение: полный файл HeaderExchange.cs для сверки 60

1.1. Сопровождение существующего журнала

1. Производственная ситуация

Обеспечить обмен CSV в существующем настольном журнале своей предметной области. Сохранить работавшие функции, воспроизвести неисправность, установить причину, внести изменение и проверить обновление на копии данных. Образец «Сервисный журнал учебного центра» объясняет способ; назначенная область и её записи остаются индивидуальными.

2. Исходный результат

Использовать ранее принятый WPF-проект своей области. При отсутствии пригодного проекта получить предоставленную основу ServiceDesk-source.zip и изменить только предметные данные и подписи. Повторную разработку уже принятого окна, встроенного списка и обработчиков не выполнять. Демонстрацию читать как разбор; обязательный проект выполняется один — в назначенной области.

3. Маршруты групп

ДКИП-4XXпрог выполняет основной маршрут: диагностика, исправление обмена, предметный отбор, регрессия и передача с откатом. VДКИП-31Хпрог и ДКИП-3XXпрог после основного результата выполняют расширение «Обмен по именам столбцов и диагностика несовместимости». Если основной результат принят ранее, подтвердить его регрессией и перейти к новому изменению.

1.2. Обязательные функции и ограничения

1. Структура файла

Заголовок строго Id;Item;Owner;Due;Status. Каждая запись содержит пять полей через точку с запятой. Id — целое число; Item — объект; Owner — ответственный; Due — дата dd.MM.yyyy; Status — «Открыта» либо «Завершена». Предметная область меняет значения полей, но не контракт обмена.

2. Импорт и диагностика

Принять корректные строки частично неверного файла. Для каждой пропущенной записи показать физический номер строки с учётом заголовка и причину. Проверять пять полей, целый Id, непустые Item/Owner, дату dd.MM.yyyy и допустимый статус. При неверном заголовке или недоступном файле сохранить ранее загруженный список.

3. Экспорт и просмотр

Записывать заголовок Id;Item;Owner;Due;Status и дату dd.MM.yyyy в UTF-8. Повторный импорт должен сохранять Id, Item, Owner, Due и Status каждой принятой записи. Отбор «Для оператора» меняет только представление; полный список остаётся источником экспорта. Разделитель и перевод строки внутри текстового поля в основном контракте запрещены и должны вызывать понятный отказ до записи.

4. Дата и правило области

Предусмотреть в окне поле «Контрольная дата». Для приёмки установить 01.10.2026 без изменения часов Windows. Просроченной считать только открытую запись со сроком раньше выбранного дня; срок в сам день проверки не просрочен. Особое условие и ожидаемый порядок Id взять из карточки своего варианта.

1.3. Диагностика и самостоятельная работа

1. D1–D3

На предоставленной CSV-заготовке изучить три дефекта: неполная строка прекращает импорт; экспорт нельзя повторно прочитать; завершённая заявка попадает в просрочку. Исправить их в рабочей копии своего проекта. Для каждого зафиксировать вход, действия, ожидаемое и фактическое поведение, причину и повторную проверку. Если исправления приняты ранее, повторить проверки без повторной реализации.

2. Отбор своего варианта

Реализовать правило назначенной области в OperatorView.Select. Использовать собственный основной CSV и контрольный CSV из карточки. Проверить фильтр, порядок Id и границы условия. Готовый образец решает только демонстрационную область; его ответ не подменяет решение назначенного варианта.

1.4. Дополнительный результат длинной практики

1. Новое правило обмена

Для VДКИП-31Хпрог и ДКИП-3ХХпрог добавить явно включаемый режим чтения столбцов по именам. Поддержать перестановку пяти известных столбцов при сохранении прежнего канонического CSV. Неизвестный, отсутствующий или повторённый столбец отклонять до изменения списка. Экспорт всегда формирует исходный канонический порядок.

2. Новые доказательства

На своей области подготовить переставленный заголовок и соответствующие ему значения строк, ошибочный заголовок и частично неверный файл. Показать равенство принятых данных до и после преобразования, правильный физический номер ошибки и сохранение прежнего списка при отказе. D1–D3 повторить только как регрессию.

1.5. Приёмка обновления

1. Набор проверок

Подготовить правильный файл, неполную строку, нецелый Id, неверную дату, пустой Item или Owner, неизвестный статус, файл только с заголовком, неверный заголовок, отсутствующий файл и текстовое поле с разделителем. До запуска для каждого входа записать ожидание, после запуска — фактические записи и предупреждения.

2. Сохранность данных

После исправления экспортировать корректный набор и импортировать его в чистой копии. Сравнить Id, Item, Owner, Due, Status и порядок строк, затем повторить исходный сценарий со встроенными данными. Проверить особое правило своей предметной области отдельно от D1–D3.

3. Обновление и откат

Сохранить отдельно прежнюю программу, исходный CSV и его резервную копию. Запустить обновление на копии данных, проверить основные действия и предметное правило, затем восстановить прежнюю программу и исходный CSV. Сравнить восстановленный файл с исходником и повторить ранее работавший сценарий. Не перезаписывать исходный CSV первым экспортом.

4. Наблюдаемый результат

Сохранить реальные снимки состояния до исправления, предупреждения со строкой, принятого набора, предметной выборки, полного списка, повторного импорта и отката. Для каждого снимка указать вход и действие, которое он подтверждает.

1.6. Сдача результата

1. Программный проект

Представить проект своей области, исходные и контрольные CSV, запускаемый пакет, инструкцию оператору и протокол проверок. В самом отчёте указать доступную проверяющему ссылку на репозиторий нужного проекта.

2. Документы

Подготовить отчёт по предоставленному шаблону, индивидуальное задание, аттестационный лист, справку и презентацию по формам, выданным руководителем. Договор о практической подготовке оформляется по выданной организацией форме. Персональные данные, сроки подписания и подписантов брать из полученных документов; отсутствующую форму запросить у руководителя.

3. Самопроверка перед передачей

Проверить открытие ссылки из отчёта, комплектность запускаемой папки, наличие исходного CSV и резервной копии, соответствие снимков фактическим действиям и выполнение маршрута своей группы. В выводе назвать выполненные проверки и оставшиеся ограничения.

2. Файлы для выполнения и сверки

1. Входной проект

ServiceDesk-source.zip — предоставленная WPF-основа без CSV. Нужна при отсутствии принятого исходного проекта. ServiceDesk-csv-draft.zip — демонстрационная заготовка с намеренными D1–D3; использовать для воспроизведения и сравнения, не как второй обязательный проект.

2. Данные

demo.csv содержит две корректные записи и неполную физическую строку 4; demo-valid.csv — две корректные записи; demo-operator.csv — шесть записей для демонстрационного отбора. В карточке назначенной области находятся variant-XX.csv и variant-check-XX.csv. Не менять назначенную область на демонстрационную.

3. Сверка и расширение

ServiceDesk-fixed-example.zip — исправленный демонстрационный проект с датой, отбором и адаптером заголовков. Открывать для сверки после анализа. demo-reordered.csv и demo-bad-header.csv нужны в расширенном маршруте. Архивы распаковывать в разные папки, не складывать одинаково названные классы в один проект.

4. Форма отчёта

report-template.md — пустой шаблон содержательной части отчёта. Перенести его разделы в выданную форму Word, заполнить собственными данными и реальными результатами. Организационные формы предоставляет руководитель; шаблон отчёта их не заменяет.

ServiceDesk-source.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-source.zip

ServiceDesk-csv-draft.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-csv-draft.zip

ServiceDesk-fixed-example.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-fixed-example.zip

demo.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo.csv

demo-valid.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-valid.csv

Скачать демонстрационный CSV для отбора оператору:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-operator.csv

demo-reordered.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-reordered.csv

demo-bad-header.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-bad-header.csv

Шаблон отчёта:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/report-template.md

3.1. От исходного приложения к проверенному обновлению

1. Один рабочий проект

Выбрать ранее принятый проект своей области и сохранить его копию в Source. Если проекта нет, распаковать предоставленную основу в Source и адаптировать подписи и встроенные данные. Создать Work как рабочую копию. Демонстрационный ServiceDesk служит чтению и сверке, его отдельная разработка перед своей областью не требуется.

2. Задача сопровождения

Оператор сервисного журнала умеет просматривать встроенные заявки, но хочет обмениваться CSV. Сначала докажите исправность окна, затем воспроизведите ошибки обмена. Исправлять их по отдельности: так повторная проверка показывает причину улучшения. После общей диагностики добавить выборку для оператора и выполните собственное предметное правило.

3. Что считать рабочей копией

Распаковать исходный проект по ссылке под заголовком в папку Source. Для экспериментов создать её копию Work. Не менять Source. CSV-заготовка нужна для сравнения и воспроизведения; исправленный пример открывать для сверки после собственного исправления. Это три версии одного приложения с одинаковыми именами классов, поэтому их исходники нельзя складывать в один проект.

4. Среда

В проверенной среде используются Windows, Visual Studio и .NET SDK 10.0.401. В файле ServiceDesk.csproj указан net10.0-windows и UseWPF=true. SDK компилирует проект; Desktop Runtime выполняет WPF-приложение. Если Visual Studio не поддерживает этот SDK или не видит WPF-компоненты, остановить сборку и сообщите преподавателю фактическую ошибку. Не менять целевую платформу наугад.

ServiceDesk-source.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-source.zip

ServiceDesk-csv-draft.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-csv-draft.zip

ServiceDesk-fixed-example.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-fixed-example.zip

3.2. Открытие и контроль исходной версии

1. Открыть проект

Запустить Visual Studio. На стартовом окне выбрать «Открыть проект или решение» (Open a project or solution), найти распакованный ServiceDesk.csproj. Через «Вид → Обозреватель решений» (View → Solution Explorer) открыть дерево проекта. Разверните MainWindow.xaml: вложенный MainWindow.xaml.cs — код окна. XAML описывает расположение и привязки; обработчик C# отвечает на действие пользователя.

2. Собрать и запустить

В верхней панели выбрать Debug. Нажать «Сборка → Собрать решение» (Build → Build Solution), затем F5. Сначала исправляют первую ошибку сборки, а не последующие следствия. В окне нажать «Тестовые данные»: появятся проектор и ноутбук. Нажать «Просрочено»: одна открытая заявка просрочена, завершённая исключена. Повторная загрузка не должна удваивать список. В поле «Контрольная дата» установить 01.10.2026. Системную дату не менять.

3. Разобрать событие

Открыть MainWindow.xaml во вкладке XAML. У кнопки есть Click="LoadTestData_Click". В коде окна найти этот метод через Ctrl+F. Он получает List<Ticket>, назначает его ItemsSource и меняет TextBlock StatusText. DataGrid читает значения свойств по Binding. InitializeComponent создаёт элементы, определённые в XAML; без него имена TicketsGrid и StatusText не готовы к использованию.

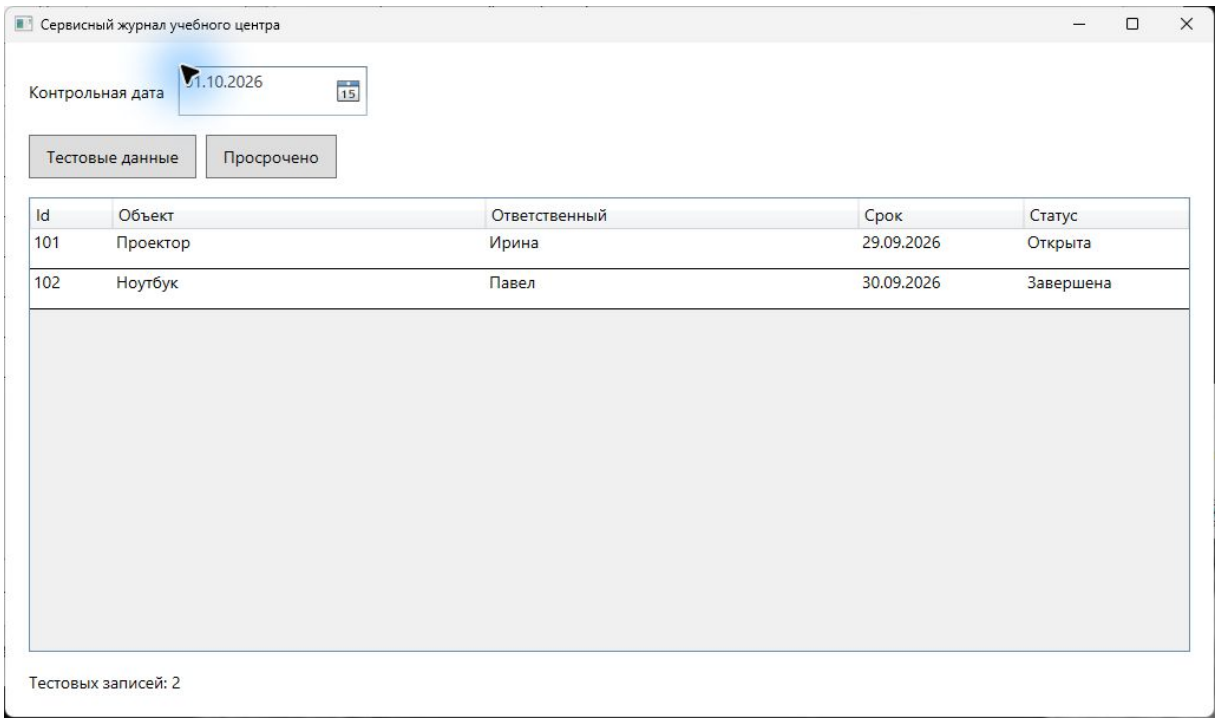


Рисунок 1 — Встроенные данные при выбранной дате

На работающем окне выставлена дата 01.10.2026; две встроенные заявки доступны до подключения CSV. В демонстрации выбранная дата даёт сроки 29.09.2026 и 30.09.2026.

ServiceDesk-source.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-source.zip

3.3. Контрольная дата как входное значение

1. Назначение

Контрольная дата — день, относительно которого проверяются сроки. DateTime.Today зависит от дня запуска и не позволяет воспроизвести сохранённый контрольный ответ. CheckDate читает выбранную дату из DatePicker; одинаковый день передаётся счётчику и OperatorView.Select.

2. Где изменить

В уже предоставленных Source и Draft поле ControlDate находится в MainWindow.xaml, значение назначается в конструкторе MainWindow, а свойство CheckDate — в MainWindow.xaml.cs. При переносе в ранее принятый проект добавить поле, событие ControlDate_Changed и свойство из полных файлов основы; заменить передачу DateTime.Today в обработчиках на CheckDate. Не менять параметр today внутри чистых методов.

3. Проверка

В окне установить 01.10.2026. Открыть demo-operator.csv и получить три демонстрационные записи 105, 101, 103. Установить 04.11.2026 и повторить отбор: получится пять открытых просроченных записей. Вернуть 01.10.2026. Проверка меняет только значение в приложении. Проверка. Системные часы прежние; результат меняется после повторного действия с новой выбранной датой.

3.4. Подключение CSV без потери исходной проверки

1. Получить заготовку

Скачать CSV-заготовку по ссылке под заголовком. Распаковать её в отдельную папку Draft, открыть её ServiceDesk.csproj в отдельном окне Visual Studio. В Work сохранить исходные App.xaml и ServiceDesk.csproj. Из Draft заменить полностью только App.xaml.cs, MainWindow.xaml и MainWindow.xaml.cs. Это согласованное изменение: новые Click, обработчики и чтение CSV появляются вместе. Не добавлять второй MainWindow.xaml.cs рядом с первым.

2. Проверить объединённую версию

Собрать Work и запустить. Должны одновременно присутствовать «Тестовые данные», «Открыть CSV», «Экспорт CSV», «Просрочено». Повторить встроенные данные: два объекта и одна просрочка. Существующий сценарий сохраняется, потому что TestData и его обработчик перенесены в CSV-заготовку. Новые функции не должны уничтожать прежнюю контрольную точку.

3. Происхождение кода

Заготовка является предоставленной версией ServiceDesk с намеренными D1–D3. Дальнейшие фрагменты берутся из исправленного демонстрационного исходника и помещаются только в точно названные участки. Стандартный код Visual Studio — InitializeComponent и сгенерированные файлы obj — вручную не редактировать.

ServiceDesk-csv-draft.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-csv-draft.zip

3.5. Входные данные и воспроизведение трёх дефектов

1. Прогноз до импорта

Скачать demo.csv и demo-valid.csv. Открыть их в текстовом редакторе. Заголовок — Id;Item;Owner;Due;Status. Разделитель — точка с запятой, дата — dd.MM.yyyy. В demo.csv две полные записи и неполная физическая строка 4 с Id=103. Номер строки относится к месту в файле, Id — к объекту. Ожидать две записи и предупреждение, а не потерю корректных заявок.

2. D1: ошибка всей загрузки

В Work нажать «Открыть CSV» и выбрать demo.csv. Зафиксировать сообщение ошибки и отсутствие результата загрузки. В MainWindow.xaml.cs найти CsvStore.Read. Поставить точку останова щелчком в левом поле перед var cells. Через F10 и «Отладка → Окна → Локальные» (Debug → Windows → Locals) проследите длину массива. У неполной строки только три поля, обращение к индексу 3 уже ошибочно; проверка должна предшествовать любому чтению недостающего поля.

3. D2 и D3 независимо от D1

Открыть demo-valid.csv. Нажать «Просрочено»: черновая версия считает две прошлые даты, включая завершённую. Экспортировать через «Экспорт CSV» в новый файл broken-export.csv и открыть его текстовым редактором: запятая и yyyy-MM-dd расходятся с контрактом. Повторный импорт этого файла не работает. Сохранить наблюдения до правки; после неё результат сравнивают с тем же входом.

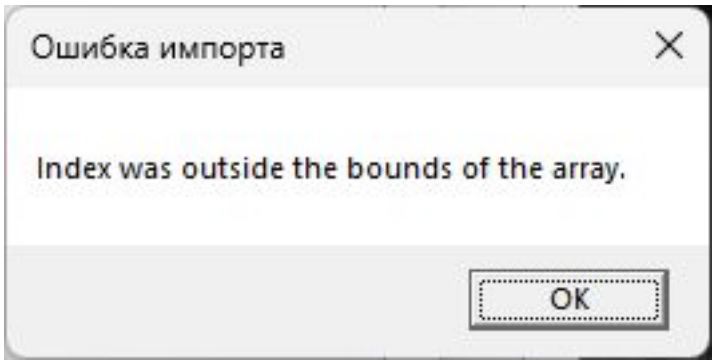


Рисунок 2 — D1 до исправления

CSV-заготовка прекращает чтение demo.csv при неполной строке; исправленная версия должна вернуть корректные записи и предупреждение.

demo.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo.csv

demo-valid.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-valid.csv

ServiceDesk-csv-draft.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-csv-draft.zip

3.6. D1: результат чтения содержит данные и предупреждения

1. Согласованная замена

Остановить отладку Shift+F5. В Work/MainWindow.xaml.cs после существующей строки internal record Ticket добавить следующую одну строку. ReadResult — новый тип результата, который хранит и корректные записи, и причины пропуска. Внутри CsvStore заменить целиком метод public static List<Ticket> Read(string path), включая его фигурные скобки, следующим методом Read. Остальные методы пока оставить прежними.

2. Код для указанного изменения

```
internal record ReadResult(List<Ticket> Tickets, List<string> Warnings);
```

3. Код для указанного изменения

```
    private const string Header = "Id;Item;Owner;Due;Status";

    public static ReadResult Read(string path)
    {
        var tickets = new List<Ticket>();
        var warnings = new List<string>();
        var lines = File.ReadAllLines(path, Encoding.UTF8);
        if (lines.Length == 0 || lines[0].TrimStart('\uFEFF') != Header)
            throw new InvalidDataException($"Ожидается заголовок {Header}");
        for (int index = 1; index < lines.Length; index++)
        {
            if (string.IsNullOrWhiteSpace(lines[index])) continue;
            var cells = lines[index].Split(';');
            string? reason = null;
            if (cells.Length != 5) reason = "ожидается пять полей";
            else if (!int.TryParse(cells[0], out _)) reason = "неверный Id";
            else if (string.IsNullOrWhiteSpace(cells[1]) ||
string.IsNullOrWhiteSpace(cells[2])) reason = "пустое обязательное поле";
            else if (!DateTime.TryParseExact(cells[3], "dd.MM.yyyy",
CultureInfo.InvariantCulture, DateTimeStyles.None, out _)) reason = "неверная
дата";
            else if (cells[4] != "Открыта" && cells[4] != "Завершена") reason
= "неверный статус";
            if (reason != null) { warnings.Add($"Строка {index + 1}:
{reason}"); continue; }
            tickets.Add(new Ticket(int.Parse(cells[0],
CultureInfo.InvariantCulture), cells[1], cells[2],
                DateTime.ParseExact(cells[3], "dd.MM.yyyy",
CultureInfo.InvariantCulture), cells[4]));
        }
        return new ReadResult(tickets, warnings);
    }
```

4. Смысл чтения

ReadAllLines получает строки в UTF-8. Заголовок проверяется до цикла. Пустые строки не являются заявками; пять полей проверяются до индексов. TryParse возвращает успех вместо исключения; TryParseExact требует однозначную дату. reason содержит первую найденную причину. continue пропускает только текущую строку. index+1 — её физический номер с учётом заголовка. Хорошая строка создаёт Ticket и остаётся доступной.

5. Завершить зависимое изменение

Внутри MainWindow заменить целиком private void LoadCsv(string path) следующим фрагментом. В App.xaml.cs заменить весь файл кодом из следующего блока. Это адаптирует запуск и режим самопроверки к ReadResult. Пока эти три связанные замены не закончены, промежуточная сборка может сообщать несовместимые типы; это не контрольная точка.

6. Код для указанного изменения

```
    private void LoadCsv(string path)
    {
        try
        {
            var result = CsvStore.Read(path);
            tickets = result.Tickets;
            TicketsGrid.ItemsSource = tickets.ToList();
            StatusText.Text = $"Загружено: {tickets.Count}; пропущено:
{result.Warnings.Count}";
            if (result.Warnings.Count > 0)
                MessageBox.Show(string.Join(Environment.NewLine,
result.Warnings), "Предупреждения импорта");
        }
        catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка импорта");
}
    }
```

7. Код для указанного изменения

```
using System.IO;
using System.Windows;
using System.Globalization;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        var args = e.Args.ToList();
        var checkDate = new DateTime(2026, 10, 1);
        int dateIndex = args.IndexOf("--date");
        if (dateIndex >= 0)
        {
            if (dateIndex + 1 >= args.Count ||
!DateTime.TryParseExact(args[dateIndex + 1],
                "yyyy-MM-dd", CultureInfo.InvariantCulture,
DateTimeStyles.None, out checkDate))
            {
                MessageBox.Show("Параметр --date требует дату yyyy-MM-dd",
"Ошибка даты");
                Shutdown(2);
                return;
            }
            args.RemoveRange(dateIndex, 2);
        }
        var runArgs = args.ToArray();
        if (runArgs.Length == 1 && runArgs[0] == "--test-built-in")
        {
            var rows = TestData.Create(checkDate);
```

7. Код для указанного изменения — продолжение

```
            Console.WriteLine($"test-records={rows.Count};
overdue={CsvStore.CountOverdue(rows, checkDate)}");
            Shutdown();
            return;
        }
        if (runArgs.Length == 2 && runArgs[0] == "--self-test")
        {
            var result = CsvStore.Read(runArgs[1]);
            var target = Path.GetTempFileName();
            try
            {
                CsvStore.Write(target, result.Tickets);
                var again = CsvStore.Read(target);
                if (again.Warnings.Count != 0 ||
!result.Tickets.SequenceEqual(again.Tickets))
                    throw new InvalidOperationException("Повторный импорт не
совпал");
                Console.WriteLine($"tickets={result.Tickets.Count};
warnings={result.Warnings.Count};
overdue={CsvStore.CountOverdue(result.Tickets, checkDate)}");
                foreach (var warning in result.Warnings)
Console.WriteLine(warning);
            }
            finally { File.Delete(target); }
            Shutdown();
            return;
        }
        new MainWindow(runArgs.Length == 2 && runArgs[0] == "--demo" ?
runArgs[1] : null, checkDate).Show();
    }
}
```

8. Проверка D1

Теперь собрать Work, запустить F5 и открыть demo.csv. Таблица должна сохранить заявки 101 и 102, сообщение — «Строка 4: ожидается пять полей», состояние — «Загружено: 2; пропущено: 1». Закрыть сообщение: строки остаются. Нажать «Тестовые данные» и проверить исходный сценарий. Экспорт пока намеренно несовместим — его исправляют далее.

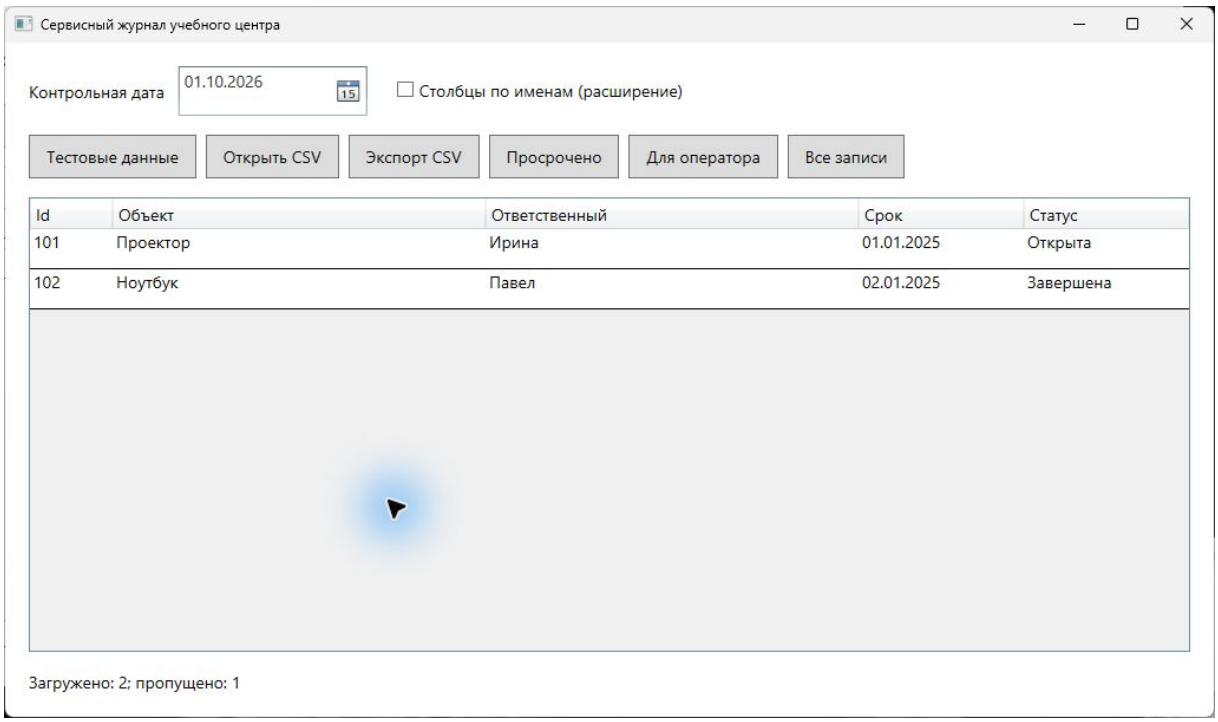


Рисунок 3 — Предупреждение относится к строке файла

Открыт demo.csv. Строки 101 и 102 приняты; предупреждение называет физическую строку 4. После закрытия сообщения данные остаются в таблице.

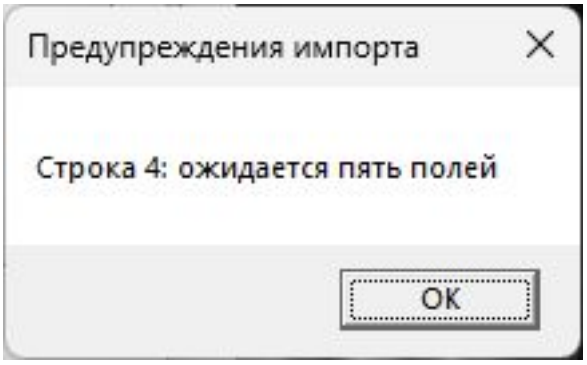


Рисунок 4 — Физическая строка ошибки

Отдельное сообщение после чтения demo.csv: строка 4 содержит меньше пяти полей.

demo.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo.csv

3.7. D2: запись соблюдает тот же контракт, что чтение

1. Точное место правки

В Work/MainWindow.xaml.cs, внутри CsvStore, заменить целиком public static void Write(string path, IEnumerable<Ticket> rows) следующим методом. Константа Header уже появилась вместе с Read: если её ещё нет, добавить private const string Header = "Id;Item;Owner;Due;Status"; непосредственно после открывающей скобки CsvStore. Не объявлять Header дважды.

2. Код для указанного изменения

```
    public static void Write(string path, IEnumerable<Ticket> tickets)
    {
        var lines = new List<string> { Header };
        foreach (var t in tickets)
        {
            if (new[] { t.Item, t.Owner, t.Status }.Any(value =>
value.Contains(';') || value.Contains('\n') || value.Contains('\r')))
                throw new InvalidDataException("В поле нельзя использовать
разделитель или перевод строки");

lines.Add($"{t.Id};{t.Item};{t.Owner};{t.Due.ToString("dd.MM.yyyy",
CultureInfo.InvariantCulture)};{t.Status}");
        }
        File.WriteAllLines(path, lines, Encoding.UTF8);
    }
```

3. Зачем две проверки

Сначала формируется весь список строк, затем File.WriteAllLines записывает его в UTF-8. ToString с InvariantCulture сохраняет dd.MM.yyyy независимо от системной культуры. В этом простом контракте нет CSV-кавычек и экранирования: точка с запятой и перевод строки внутри поля отклоняются, чтобы не создать повреждённый файл. Отказ происходит до записи содержимого.

4. Сравнить значения

Собрать, открыть demo-valid.csv, экспортировать в roundtrip.csv и открыть этот результат. В текстовом редакторе сравнить заголовок, оба Id, Item, Owner, Due и Status; одинакового количества недостаточно. Вернитесь к demo.csv: предупреждение о строке 4 должно остаться. Экспорт сохраняет загруженные корректные записи и не восстанавливает отвергнутую строку.

demo-valid.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-valid.csv

demo.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo.csv

3.8. D3: просрочка зависит от срока и состояния

1. Заменить одно правило

Внутри CsvStore заменить целиком метод CountOverdue следующим фрагментом. Не менять TestData.CountOverdue: он был правильным и нужен для сравнения. Два правила должны давать одинаковый результат на одинаковых данных.

2. Код для указанного изменения

```
    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime
today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date <
today.Date);
```

3. Разобрать выражение

Count считает только элементы с истинным условием. Статус завершения исключается до сравнения дат. Due.Date и today.Date сравнивают календарные дни без времени. Срок на контрольную дату ещё не просрочен. Параметр today позволяет проверить правило на заданную дату без изменения часов Windows.

4. Граничная проверка

Собрать и открыть demo-valid.csv: после «Просрочено» ожидается 1. В копии CSV создать открытую вчера, завершённую вчера и открытую сегодня — ожидается 1; заменить вчера на завтра — ожидается 0. Затем снова загрузите demo.csv и нажать «Тестовые данные»: два разных источника данных должны оставаться рабочими.

demo-valid.csv:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-valid.csv

3.9. Отбор для оператора без потери исходного списка

1. Демонстрационная задача

В отдельной демонстрационной области оператору нужны незавершённые просроченные заявки по сроку, затем по Id. Полный импорт должен оставаться в памяти и экспортироваться независимо от того, какую выборку сейчас видит оператор. Так пользовательский просмотр не превращается в потерю данных.

2. Добавить правило и события

В Work/MainWindow.xaml.cs между CsvStore и MainWindow добавить класс OperatorView. Внутри MainWindow добавить два обработчика из следующего фрагмента. В MainWindow.xaml заменить только WrapPanel Grid.Row="1" с кнопками на приведённый WrapPanel; строку с ControlDate и её событие сохранить. Это одно связанное изменение; собрать после всех вставок.

3. Код для указанного изменения

```
internal static class OperatorView
{
    public static List<Ticket> Select(IEnumerable<Ticket> rows, DateTime
today) =>
        rows.Where(t => t.Status == "Открыта" && t.Due.Date < today.Date)
            .OrderBy(t => t.Due).ThenBy(t => t.Id).ToList();
}
```

4. Код для указанного изменения

```
    private void SelectForOperator_Click(object sender, RoutedEventArgs e)
    {
        var selected = OperatorView.Select(tickets, CheckDate);
        TicketsGrid.ItemsSource = selected;
        StatusText.Text = $"Для оператора: {selected.Count}; всего:
{tickets.Count}";
    }

    private void ShowAll_Click(object sender, RoutedEventArgs e)
    {
        TicketsGrid.ItemsSource = tickets.ToList();
        StatusText.Text = $"Все записи: {tickets.Count}";
    }
```

5. Код для указанного изменения

```
    <WrapPanel Grid.Row="1" Margin="0,0,0,8">
      <Button Content="Тестовые данные" Click="LoadTestData_Click" />
      <Button Content="Открыть CSV" Click="OpenCsv_Click" />
      <Button Content="Экспорт CSV" Click="SaveCsv_Click" />
      <Button Content="Просрочено" Click="CountOverdue_Click" />
      <Button Content="Для оператора" Click="SelectForOperator_Click" />
      <Button Content="Все записи" Click="ShowAll_Click" />
    </WrapPanel>
```

6. Устройство выборки

Where оставляет нужные записи, OrderBy задаёт главный порядок, ThenBy разрешает равенство срока, ToList создаёт отдельный список для таблицы. tickets не заменяется выборкой. «Все записи» возвращает полный набор. WrapPanel переносит кнопки при недостатке ширины окна. При реализации своего варианта меняется только Select, а загрузка, экспорт и восстановление просмотра сохраняются.

7. Проверить действие

Собрать и открыть demo-operator.csv. В поле «Контрольная дата» установить 01.10.2026, нажать «Для оператора»: ожидаемые Id — 105, 101, 103. Экспортировать в новый файл и повторно импортировать: сохранить все шесть записей. Кнопка «Все записи» также должна показать шесть строк. Затем изменить выбранную дату и заново нажать отбор; после проверки вернуть 01.10.2026.

3.9. Отбор для оператора без потери исходного списка — продолжение

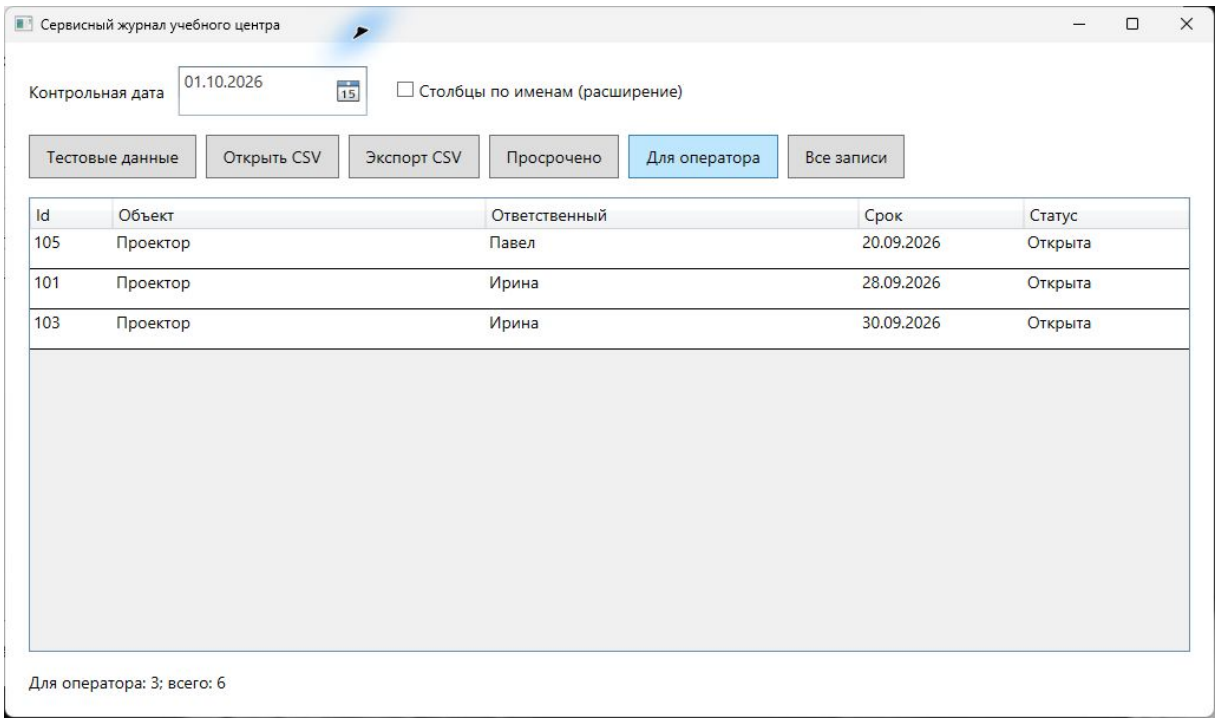


Рисунок 5 — Отбор на контрольную дату

При явно заданной дате 01.10.2026 отображаются Id 105, 101, 103; исходный список содержит шесть записей.

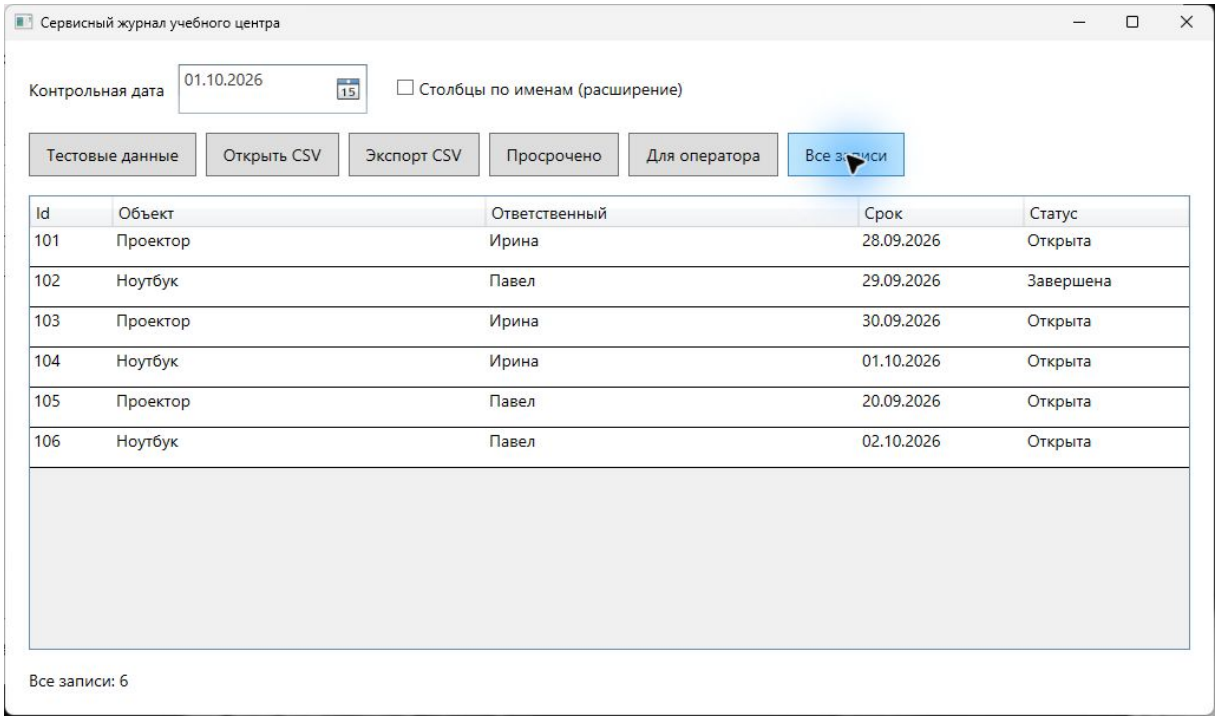


Рисунок 6 — Полный список после отбора

Действие «Все записи» восстанавливает шесть исходных строк; экспорт использует полный список.

Скачать демонстрационный CSV для отбора оператору:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/demo-operator.csv

3.10. Как перенести правило на индивидуальную область

1. Собрать условие до кода

Из карточки выписать статус, ответственного, границы даты, порядок и ограничение числа записей. Взять фактические имена ответственных из своего контрольного CSV. Перевести каждую часть в отдельное условие; не копировать контрольные Id демонстрации в код.

2. Сравнения и календарные границы

t.Due.Date < today.Date — срок раньше выбранного дня; <= включает сам день. Диапазон задаётся двумя сравнениями через &&. today.Date.AddDays(-N) означает день за N суток до контрольного. «Не менее N дней назад» включает границу, «строго менее N дней назад» требует открытой нижней границы. Для статуса и Owner использовать сравнение строк с точными значениями из CSV.

3. Порядок и ограничение

Where отбирает записи. OrderBy задаёт первый ключ, ThenBy — следующий; OrderByDescending и ThenByDescending меняют направление соответствующего ключа. После сортировки Take(N) оставляет первые N. ToList создаёт отдельное представление. Исходный tickets не заменять результатом.

4. Самостоятельная проверка

Вручную отметить подходящие строки своего контрольного CSV и упорядочить их по условию. Сравнить с порядком Id в карточке на 01.10.2026. Добавить равный срок, день на границе и запись с противоположным статусом; заранее определить ожидаемое включение. Этот разбор не предоставляет готовый метод для назначенной области.

3.11. Регрессионная матрица и диагностика ошибок

1. Набор проверок

На копиях входа проверить: верный файл; неполную строку после хорошей; неверные Id, дату и статус; пустые Item/Owner; файл только с заголовком; неверный заголовок; отсутствующий файл; строку с лишним разделителем. Для каждого случая указать вход, действие, ожидаемые записи/сообщение, фактический результат и вывод. Header-only даёт пустой список без предупреждений; неверный заголовок запрещает загрузку; ошибочная отдельная запись не уничтожает хорошие.

2. Сохранение после отказа

Загрузите верный CSV, затем попробовать файл с неверным заголовком. Благодаря назначению tickets только после успешного Read прежние данные остаются в памяти. Экспортировать их и сравнить все поля. Если запись файла запрещена или файл занят, окно сообщает ошибку экспорта; не считать сообщение «Файл сохранён» доказательством без проверки файла.

3. Проверка исходной функции

После всех исправлений нажать «Тестовые данные», «Просрочено», «Для оператора», «Все записи». Ожидаются две встроенные заявки, одна просрочка и восстановление двух строк. После повторного CSV-импорта выборка должна строиться по новому набору. Выполните полный цикл импорта и экспорта ещё раз: проверка отсекает случайные изменения за пределами D1–D3.

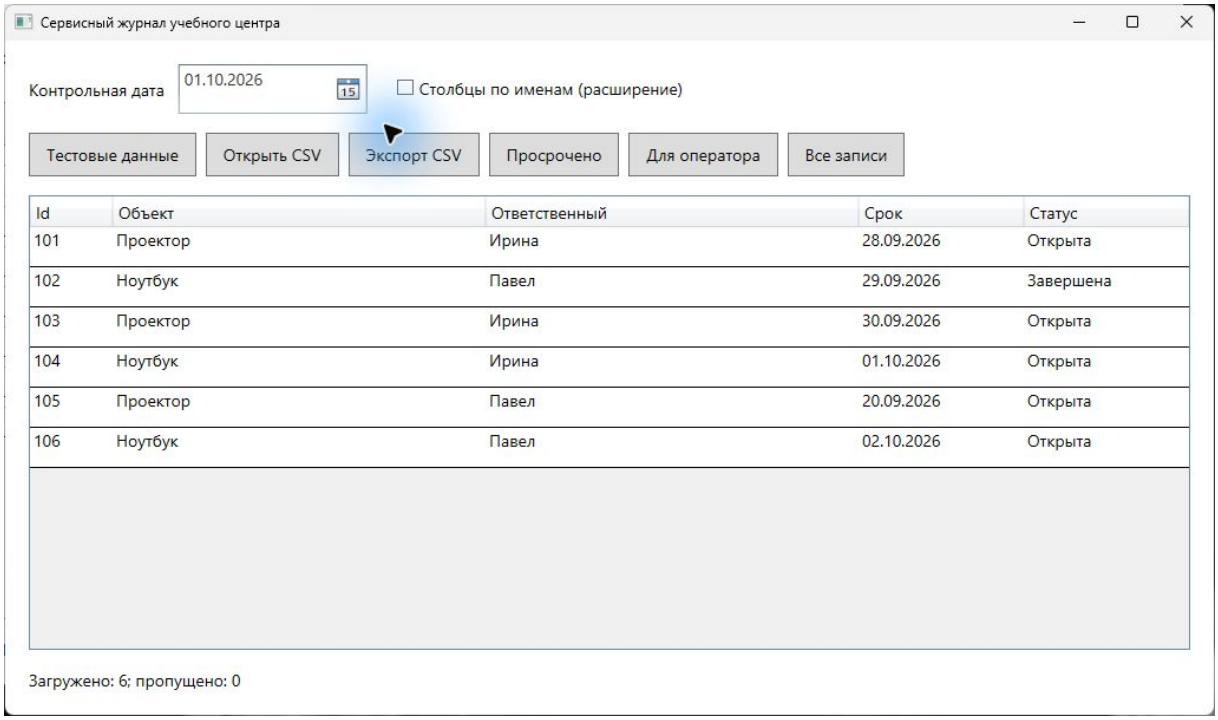


Рисунок 7 — Повторное чтение полного экспорта

После отбора экспортирован полный демонстрационный список. Повторное чтение этого CSV возвращает шесть записей со всеми пятью значениями.

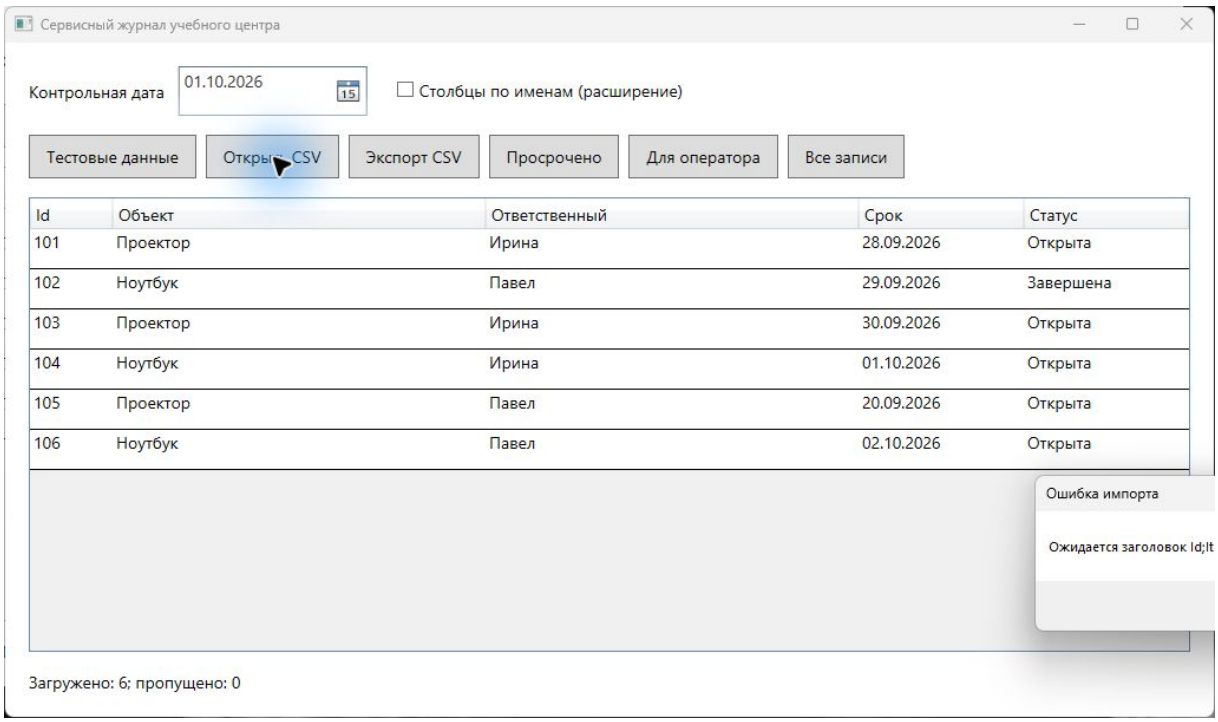


Рисунок 8 — Ошибка заголовка сохраняет прежние данные

При отказе чтения demo-bad-header.csv прежние шесть строк остаются доступными. Сообщение указывает ожидаемый контракт.

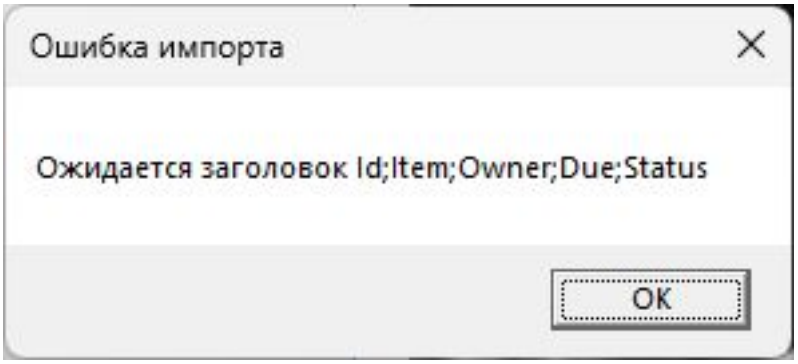


Рисунок 9 — Причина отказа импорта

Сообщение называет ожидаемые имена пяти столбцов. Прежний список остаётся доступным после закрытия диалога.

3.12. Release, резервная копия и передача обновления

1. Подготовить запускаемую версию

Остановить отладку Shift+F5. В верхней панели Visual Studio выбрать Release и «Сборка → Собрать решение». В обозревателе решений щёлкнуть проект правой кнопкой и выбрать «Открыть папку в проводнике» (Open Folder in File Explorer). Открыть bin/Release/net10.0-windows. Скопировать содержимое этой папки целиком в новую папку Delivery/v1.2; копия одного exe недостаточна. Не брать obj. На целевом Windows должен быть .NET Desktop Runtime 10, соответствующий архитектуре сборки; этот выпуск зависит от установленной среды и не содержит её установщик.

2. Подготовить резервирование

В отдельной учебной папке Delivery сохранить прежнюю проверенную версию в v1.1, рабочий CSV — в data/work.csv, резервную копию этого CSV — в backup/work-before.csv. Программу и данные храните отдельно. До замены закрыть приложение и сравнить текст рабочей и резервной копии. Не выполнять упражнение на реальных данных организации.

3. Передать и проверить

Запустить exe из v1.2 двойным щелчком, открыть копию data/work.csv. Проверить предупреждение, две сохранённые записи, счётчик, встроенные данные и предметную выборку. Экспортировать в data/export-check.csv и повторно импортировать. Записать версию, дату, среду, действия и реальные сообщения. Оператор получает понятную инструкцию с названиями кнопок и действиями при ошибке заголовка или чтения.

3.13. Проверенный откат и работа по своему варианту

1. Выполнить откат

Закрыть v1.2. Сохранить диагностический экспорт отдельно, заменить учебный data/work.csv копией backup/work-before.csv. Запустить прежнюю программу из v1.1 на восстановленной копии. Сравнить исходное содержимое CSV и повторить сценарий, который работал до обновления. Намеренные дефекты прежней CSV-версии при этом остаются — откат возвращает прежнее состояние, а не исправляет его. Затем снова закрыть приложение и запустить v1.2; резервная копия остаётся нетронутой.

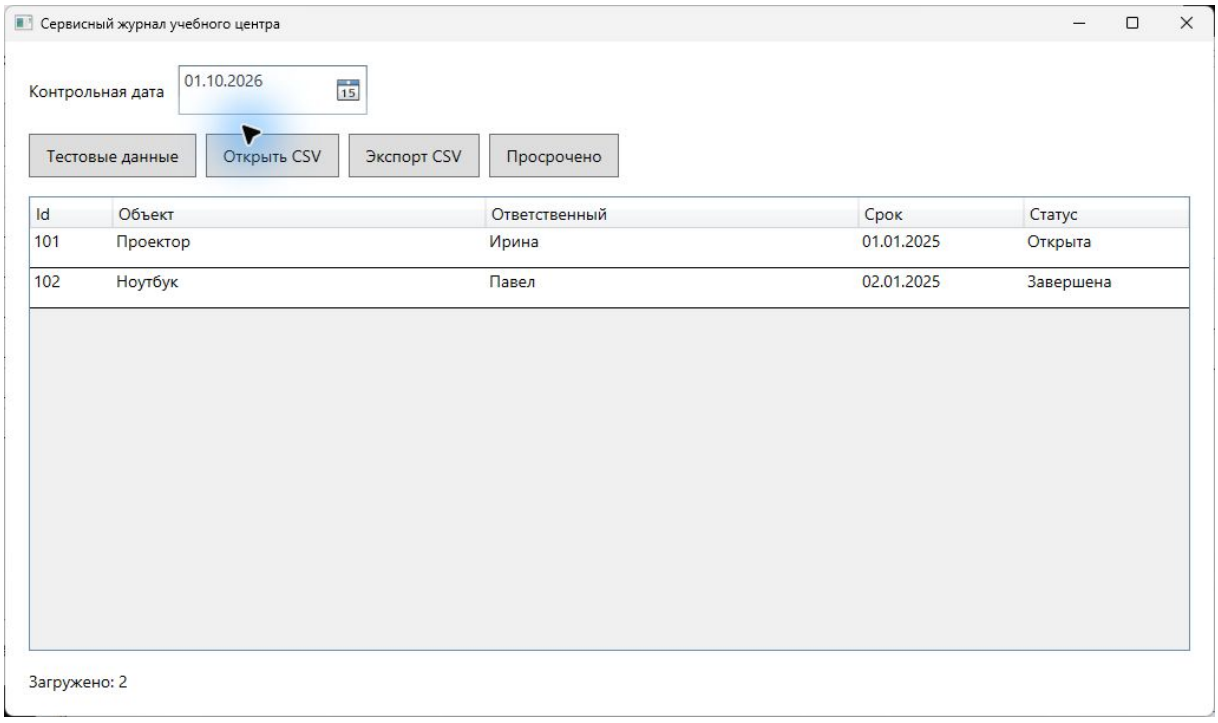


Рисунок 10 — Прежняя версия на восстановленном CSV

В учебной папке восстановлена побайтовая копия demo-valid.csv; запущена сохранённая прежняя CSV-версия. Две исходные записи снова доступны. Намеренные дефекты этой версии сохраняются.

3. Адаптировать область

Открыть карточку своего варианта: все нужные ссылки повторены там. В проверенной Work-копии поменять Window.Title и подписи сущностей, в TestData.Create — значения объектов и ответственных. Сохранить пять полей контракта и обработчик встроенных данных. Сначала повторить исходную проверку, затем загрузите variant-XX.csv: две хорошие записи и предупреждение о строке 4.

4. Реализовать самостоятельное правило

После D1–D3 реализовать правило карточки в OperatorView.Select. Для фильтра использовать Where, для порядка OrderBy/ThenBy, для ограничения Take; элементы цепочки возвращают набор записей. Как и в демонстрации, не изменять tickets и не превращать экспорт в экспорт выборки. Отдельный контрольный CSV назначенного варианта содержит шесть хороших записей. Сверить указанный порядок Id на контрольную дату; затем проверить границы и полный экспорт. Это собственная задача, исправленный пример решает только демонстрационную область.

5. Описать передачу

Объяснить причину каждого исправления, ограничения CSV, правило области, результаты проверки и процедуру отката. Использовать общий шаблон отчёта с собственной датой, версией и реальными скриншотами. При сдаче проекта приложить ссылку на его репозиторий.

ServiceDesk-fixed-example.zip:
https://olgakraven.github.io/2026-2027-09.02.07-prog-pp-pm.04-sem07-part01/downloads/ServiceDesk-fixed-example.zip

3.14. Расширенный маршрут: столбцы по именам

1. Вход и новый результат

Основной маршрут уже принят: чтение, запись, дата и отбор работают. Сохранить рабочую версию и данные перед новым изменением. Требуется читать те же поля при другой очередности столбцов, сохранять старый формат и прежнюю диагностику. Выполнить этот маршрут для VДКИП-31Хпрог и ДКИП-3ХХпрог.

2. Демонстрационный прогноз

Открыть demo-reordered.csv в редакторе: порядок Owner;Status;Id;Due;Item. Сравнить значения с demo-operator.csv. Строгое чтение обязано отклонить такой заголовок, а новый режим — восстановить те же шесть объектов. demo-bad-header.csv дважды называет Due и не содержит Status; оба режима должны его отклонить.

3. Адаптер вместо повторного исправления D1–D3

Добавить отдельный файл HeaderExchange.cs из исправленного демонстрационного архива. Метод проверяет ровно пять известных уникальных имён, строит индексы канонического порядка и переставляет значения каждой полной строки. Пустые и неполные строки сохраняются на своих физических местах. Нормализованный временный файл передаётся уже принятому CsvStore.Read; finally удаляет только этот временный файл. Исходный CSV не изменяется.

4. HeaderExchange.cs — полный демонстрационный адаптер

```
using System.IO;
using System.Text;

namespace ServiceDeskWpf;

// Optional extension: adapt the order, then reuse the accepted strict reader.
internal static class HeaderExchange
{
    private static readonly string[] Columns = ["Id", "Item", "Owner", "Due",
"Status"];

    public static ReadResult Read(string path)
    {
        var lines = File.ReadAllLines(path, Encoding.UTF8);
        if (lines.Length == 0) throw new InvalidDataException("Отсутствует
заголовок CSV");
        var header = lines[0].TrimStart('\uFEFF').Split(';');
        if (header.Length != Columns.Length ||
header.Distinct(StringComparer.Ordinal).Count() != Columns.Length ||
            Columns.Any(name => !header.Contains(name,
StringComparer.Ordinal)))
            throw new InvalidDataException("Заголовок должен содержать Id,
Item, Owner, Due, Status по одному разу");
        var order = Columns.Select(name => Array.IndexOf(header,
name)).ToArray();
        var normalized = new List<string> { string.Join(';', Columns) };
        foreach (var line in lines.Skip(1))
        {
            if (string.IsNullOrWhiteSpace(line)) { normalized.Add(line);
continue; }
            var cells = line.Split(';');
            // Preserve malformed rows and physical line numbers for the
```

4. HeaderExchange.cs — полный демонстрационный адаптер — продолжение

```
existing validator.
            normalized.Add(cells.Length == Columns.Length ? string.Join(';',
order.Select(i => cells[i])) : line);
        }
        var temp = Path.GetTempFileName();
        try { File.WriteAllLines(temp, normalized, Encoding.UTF8); return
CsvStore.Read(temp); }
        finally { File.Delete(temp); }
    }
}
```

5. Связать с окном

В MainWindow.xaml в строку StackPanel с ControlDate добавить CheckBox с x:Name="FlexibleHeader" и подписью «Столбцы по именам (расширение)». В MainWindow.LoadCsv заменить только вызов чтения на выражение ниже. CsvStore.Write оставить прежним: экспорт всегда канонический. Собрать после добавления файла, флажка и изменения вызова.

6. Выбор режима чтения в LoadCsv

```
var result = FlexibleHeader.IsChecked == true
    ? HeaderExchange.Read(path)
    : CsvStore.Read(path);
```

7. Проверить новое поведение

Сначала открыть demo-operator.csv без флажка. Затем отметить «Столбцы по именам» и открыть demo-reordered.csv: сравнить все пять значений шести записей. Экспортировать в новый файл, снять флажок и прочитать экспорт строгим режимом. После попытки открыть demo-bad-header.csv прежние шесть записей должны остаться в памяти.

3.14. Расширенный маршрут: столбцы по именам — продолжение

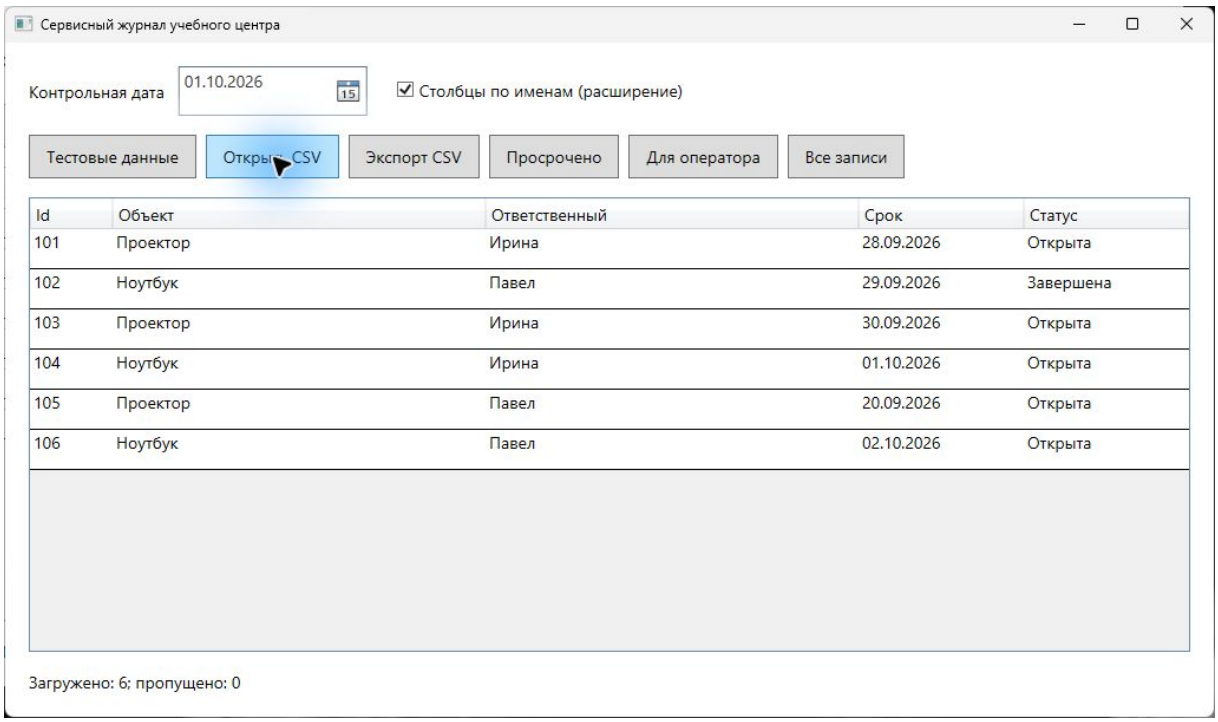


Рисунок 11 — Переставленные столбцы приняты новым режимом

Включено чтение по именам. В окне показаны те же шесть демонстрационных записей при новом порядке столбцов входного файла.

9. Перенести на назначенную область

В копии собственного контрольного CSV переставить заголовок и значения каждой строки одинаковой перестановкой. Не менять поля и значения объектов. Подготовить копию с повторным именем и копию с неполной строкой между корректными. Описать правило обмена, ожидаемое поведение, сравнение полей и физические номера предупреждений. Повторить базовые проверки как регрессию. Проверка. Старый и новый вход дают одинаковые записи; ошибочный заголовок не меняет состояние; экспорт читается прежним строгим режимом.

4. Полные файлы для сверки

Листинги сохранены для чтения. Исполняемый текст брать из файлов архива; печатные переносы длинной строки не переносить в код. Повторное переписывание полного образца не требуется.

4.1. Исходная версия: полный файл ServiceDesk.csproj для сверки

1. ServiceDesk.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

2. Проверка связей

Целевая платформа net10.0-windows и UseWPF=true.

4.2. Исходная версия: полный файл App.xaml для сверки

1. App.xaml

```
<Application x:Class="ServiceDeskWpf.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
  <Application.Resources />
</Application>
```

2. Проверка связей

Имя ServiceDeskWpf.App совпадает с кодом приложения.

4.3. Исходная версия: полный файл App.xaml.cs для сверки

1. App.xaml.cs

```
using System.Windows;
using System.Globalization;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        var args = e.Args.ToList();
        var checkDate = new DateTime(2026, 10, 1);
        int dateIndex = args.IndexOf("--date");
        if (dateIndex >= 0)
        {
            if (dateIndex + 1 >= args.Count ||
!DateTime.TryParseExact(args[dateIndex + 1],
                "yyyy-MM-dd", CultureInfo.InvariantCulture,
DateTimeStyles.None, out checkDate))
            {
                MessageBox.Show("Параметр --date требует дату yyyy-MM-dd",
"Ошибка даты");
                Shutdown(2);
                return;
            }
            args.RemoveRange(dateIndex, 2);
        }
        var runArgs = args.ToArray();
        if (runArgs.Length == 1 && runArgs[0] == "--self-test")
        {
            var rows = TestData.Create(checkDate);
            Console.WriteLine($"test-records={rows.Count};
```

1. App.xaml.cs — продолжение

```
overdue={TestData.CountOverdue(rows, checkDate)}");
            Shutdown();
            return;
        }
        new MainWindow(runArgs.Length == 1 && runArgs[0] == "--demo",
checkDate).Show();
    }
}
```

2. Проверка связей

Разбор параметра даты и запуск нужного MainWindow.

4.4. Исходная версия: полный файл MainWindow.xaml для сверки

1. MainWindow.xaml

```
<Window x:Class="ServiceDeskWpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Сервисный журнал учебного центра" Width="1040" Height="610"
MinWidth="800" MinHeight="480"
        Background="White" FontFamily="Segoe UI" FontSize="14"
Language="ru-RU">
  <Window.Resources>
    <Style TargetType="Button">
      <Setter Property="Padding" Value="14,8" />
      <Setter Property="Margin" Value="0,0,8,8" />
      <Setter Property="MinHeight" Value="36" />
    </Style>
  </Window.Resources>
  <Grid Margin="20">
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition Height="Auto" />
      <RowDefinition Height="*" />
      <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <StackPanel Orientation="Horizontal" Margin="0,0,0,16">
      <TextBlock Text="Контрольная дата" VerticalAlignment="Center"
Margin="0,0,12,0" />
      <DatePicker x:Name="ControlDate" Width="160" SelectedDateFormat="Short"
                  SelectedDateChanged="ControlDate_Changed"
AutomationProperties.Name="Контрольная дата" />
    </StackPanel>
    <WrapPanel Grid.Row="1" Margin="0,0,0,8">
      <Button Content="Тестовые данные" Click="LoadTestData_Click" />
      <Button Content="Просрочено" Click="CountOverdue_Click" />
    </WrapPanel>
```

1. MainWindow.xaml — продолжение

```
    <DataGrid x:Name="TicketsGrid" Grid.Row="2" AutoGenerateColumns="False"
IsReadOnly="True"
              CanUserAddRows="False" RowHeight="34" HeadersVisibility="Column"
 GridLinesVisibility="Horizontal">
      <DataGrid.Columns>
        <DataGridTextColumn Header="Id" Binding="{Binding Id}" Width="70" />
        <DataGridTextColumn Header="Объект" Binding="{Binding Item}" Width="*"
 />
        <DataGridTextColumn Header="Ответственный" Binding="{Binding Owner}"
Width="*" />
        <DataGridTextColumn Header="Срок" Binding="{Binding Due,
StringFormat=dd.MM.yyyy}" Width="140" />
        <DataGridTextColumn Header="Статус" Binding="{Binding Status}"
Width="140" />
      </DataGrid.Columns>
    </DataGrid>
    <TextBlock x:Name="StatusText" Grid.Row="3" Margin="0,16,0,0"
Text="Загрузить встроенные данные для первой проверки" />
  </Grid>
</Window>
```

2. Проверка связей

Имена ControlDate, TicketsGrid, StatusText и обработчики Click совпадают с кодом.

4.5. Исходная версия: полный файл MainWindow.xaml.cs для сверки

1. MainWindow.xaml.cs

```
using System.Windows;

namespace ServiceDeskWpf;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string
 Status);

internal static class TestData
{
    public static List<Ticket> Create(DateTime today) =>
    [
        new Ticket(101, "Проектор", "Ирина", today.AddDays(-2), "Открыта"),
        new Ticket(102, "Ноутбук", "Павел", today.AddDays(-1), "Завершена")
    ];

    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime
today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date <
today.Date);
}

public partial class MainWindow : Window
{
    private List<Ticket> tickets = new();
    private DateTime CheckDate => ControlDate.SelectedDate!.Value.Date;

    public MainWindow(bool showTestData = false, DateTime? checkDate = null)
    {
        InitializeComponent();
        ControlDate.SelectedDate = checkDate ?? new DateTime(2026, 10, 1);
        if (showTestData) Loaded += (_, _) => LoadTestData();
    }
```

1. MainWindow.xaml.cs — продолжение

```
    private void ControlDate_Changed(object sender,
System.Windows.Controls.SelectionChangedEventArgs e)
    {
        if (ControlDate.SelectedDate is null)
            ControlDate.SelectedDate = new DateTime(2026, 10, 1);
    }

    private void LoadTestData_Click(object sender, RoutedEventArgs e) =>
LoadTestData();

    private void LoadTestData()
    {
        tickets = TestData.Create(CheckDate);
        TicketsGrid.ItemsSource = tickets.ToList();
        StatusText.Text = $"Тестовых записей: {tickets.Count}";
    }

    private void CountOverdue_Click(object sender, RoutedEventArgs e) =>
        StatusText.Text = $"Просрочено: {TestData.CountOverdue(tickets,
CheckDate)}";
}
```

2. Проверка связей

Список, встроенные записи, выбранная дата и обработчики образуют один согласованный файл.

4.6. Исправленная версия: полный файл ServiceDesk-fixed.csproj для сверки

1. ServiceDesk-fixed.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <RootNamespace>ServiceDeskWpf</RootNamespace>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

2. Проверка связей

Сопоставить файл с проектом образца.

4.7. Исправленная версия: полный файл App.xaml для сверки

1. App.xaml

```
<Application x:Class="ServiceDeskWpf.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
  <Application.Resources />
</Application>
```

2. Проверка связей

Имя ServiceDeskWpf.App совпадает с кодом приложения.

4.8. Исправленная версия: полный файл App.xaml.cs для сверки

1. App.xaml.cs

```
using System.IO;
using System.Windows;
using System.Globalization;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        var args = e.Args.ToList();
        var checkDate = new DateTime(2026, 10, 1);
        int dateIndex = args.IndexOf("--date");
        if (dateIndex >= 0)
        {
            if (dateIndex + 1 >= args.Count ||
!DateTime.TryParseExact(args[dateIndex + 1],
                "yyyy-MM-dd", CultureInfo.InvariantCulture,
DateTimeStyles.None, out checkDate))
            {
                MessageBox.Show("Параметр --date требует дату yyyy-MM-dd",
"Ошибка даты");
                Shutdown(2);
                return;
            }
            args.RemoveRange(dateIndex, 2);
        }
        var runArgs = args.ToArray();
        if (runArgs.Length == 1 && runArgs[0] == "--test-built-in")
        {
            var rows = TestData.Create(checkDate);
```

1. App.xaml.cs — продолжение

```
            Console.WriteLine($"test-records={rows.Count};
overdue={CsvStore.CountOverdue(rows, checkDate)}");
            Shutdown();
            return;
        }
        if (runArgs.Length == 2 && runArgs[0] == "--self-test")
        {
            var result = CsvStore.Read(runArgs[1]);
            var target = Path.GetTempFileName();
            try
            {
                CsvStore.Write(target, result.Tickets);
                var again = CsvStore.Read(target);
                if (again.Warnings.Count != 0 ||
!result.Tickets.SequenceEqual(again.Tickets))
                    throw new InvalidOperationException("Повторный импорт не
совпал");
                Console.WriteLine($"tickets={result.Tickets.Count};
warnings={result.Warnings.Count};
overdue={CsvStore.CountOverdue(result.Tickets, checkDate)}");
                foreach (var warning in result.Warnings)
Console.WriteLine(warning);
            }
            finally { File.Delete(target); }
            Shutdown();
            return;
        }
        new MainWindow(runArgs.Length == 2 && runArgs[0] == "--demo" ?
runArgs[1] : null, checkDate).Show();
    }
}
```

2. Проверка связей

Разбор параметра даты и запуск нужного MainWindow.

4.9. Исправленная версия: полный файл MainWindow.xaml для сверки

1. MainWindow.xaml

```
<Window x:Class="ServiceDeskWpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Сервисный журнал учебного центра" Width="1040" Height="610"
MinWidth="800" MinHeight="480"
        Background="White" FontFamily="Segoe UI" FontSize="14"
Language="ru-RU">
  <Window.Resources>
    <Style TargetType="Button">
      <Setter Property="Padding" Value="14,8" />
      <Setter Property="Margin" Value="0,0,8,8" />
      <Setter Property="MinHeight" Value="36" />
    </Style>
  </Window.Resources>
  <Grid Margin="20">
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition Height="Auto" />
      <RowDefinition Height="*" />
      <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <StackPanel Orientation="Horizontal" Margin="0,0,0,16">
      <TextBlock Text="Контрольная дата" VerticalAlignment="Center"
Margin="0,0,12,0" />
      <DatePicker x:Name="ControlDate" Width="160" SelectedDateFormat="Short"
                  SelectedDateChanged="ControlDate_Changed"
AutomationProperties.Name="Контрольная дата" />
      <CheckBox x:Name="FlexibleHeader" Content="Столбцы по именам
(расширение)" VerticalAlignment="Center" Margin="24,0,0,0" />
    </StackPanel>
    <WrapPanel Grid.Row="1" Margin="0,0,0,8">
      <Button Content="Тестовые данные" Click="LoadTestData_Click" />
```

1. MainWindow.xaml — продолжение

```
      <Button Content="Открыть CSV" Click="OpenCsv_Click" />
      <Button Content="Экспорт CSV" Click="SaveCsv_Click" />
      <Button Content="Просрочено" Click="CountOverdue_Click" />
      <Button Content="Для оператора" Click="SelectForOperator_Click" />
      <Button Content="Все записи" Click="ShowAll_Click" />
    </WrapPanel>
    <DataGrid x:Name="TicketsGrid" Grid.Row="2" AutoGenerateColumns="False"
IsReadOnly="True"
              CanUserAddRows="False" RowHeight="34" HeadersVisibility="Column"
 GridLinesVisibility="Horizontal">
      <DataGrid.Columns>
        <DataGridTextColumn Header="Id" Binding="{Binding Id}" Width="70" />
        <DataGridTextColumn Header="Объект" Binding="{Binding Item}" Width="*"
 />
        <DataGridTextColumn Header="Ответственный" Binding="{Binding Owner}"
Width="*" />
        <DataGridTextColumn Header="Срок" Binding="{Binding Due,
StringFormat=dd.MM.yyyy}" Width="140" />
        <DataGridTextColumn Header="Статус" Binding="{Binding Status}"
Width="140" />
      </DataGrid.Columns>
    </DataGrid>
    <TextBlock x:Name="StatusText" Grid.Row="3" Margin="0,16,0,0"
Text="Загрузить встроенные данные для первой проверки" />
  </Grid>
</Window>
```

2. Проверка связей

Имена ControlDate, TicketsGrid, StatusText и обработчики Click совпадают с кодом.

4.10. Исправленная версия: полный файл MainWindow.xaml.cs для сверки

1. MainWindow.xaml.cs

```
using System.Globalization;
using System.IO;
using System.Text;
using System.Windows;
using Microsoft.Win32;

namespace ServiceDeskWpf;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string
 Status);
internal record ReadResult(List<Ticket> Tickets, List<string> Warnings);

internal static class TestData
{
    public static List<Ticket> Create(DateTime today) =>
    [
        new Ticket(101, "Проектор", "Ирина", today.AddDays(-2), "Открыта"),
        new Ticket(102, "Ноутбук", "Павел", today.AddDays(-1), "Завершена")
    ];

    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime
today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date <
today.Date);
}

internal static class CsvStore
{
    private const string Header = "Id;Item;Owner;Due;Status";

    public static ReadResult Read(string path)
    {
```

1. MainWindow.xaml.cs — продолжение

```
        var tickets = new List<Ticket>();
        var warnings = new List<string>();
        var lines = File.ReadAllLines(path, Encoding.UTF8);
        if (lines.Length == 0 || lines[0].TrimStart('\uFEFF') != Header)
            throw new InvalidDataException($"Ожидается заголовок {Header}");
        for (int index = 1; index < lines.Length; index++)
        {
            if (string.IsNullOrWhiteSpace(lines[index])) continue;
            var cells = lines[index].Split(';');
            string? reason = null;
            if (cells.Length != 5) reason = "ожидается пять полей";
            else if (!int.TryParse(cells[0], out _)) reason = "неверный Id";
            else if (string.IsNullOrWhiteSpace(cells[1]) ||
string.IsNullOrWhiteSpace(cells[2])) reason = "пустое обязательное поле";
            else if (!DateTime.TryParseExact(cells[3], "dd.MM.yyyy",
CultureInfo.InvariantCulture, DateTimeStyles.None, out _)) reason = "неверная
дата";
            else if (cells[4] != "Открыта" && cells[4] != "Завершена") reason
= "неверный статус";
            if (reason != null) { warnings.Add($"Строка {index + 1}:
{reason}"); continue; }
            tickets.Add(new Ticket(int.Parse(cells[0],
CultureInfo.InvariantCulture), cells[1], cells[2],
                DateTime.ParseExact(cells[3], "dd.MM.yyyy",
CultureInfo.InvariantCulture), cells[4]));
        }
        return new ReadResult(tickets, warnings);
    }

    public static void Write(string path, IEnumerable<Ticket> tickets)
    {
        var lines = new List<string> { Header };
```

1. MainWindow.xaml.cs — продолжение

```
        foreach (var t in tickets)
        {
            if (new[] { t.Item, t.Owner, t.Status }.Any(value =>
value.Contains(';') || value.Contains('\n') || value.Contains('\r')))
                throw new InvalidDataException("В поле нельзя использовать
разделитель или перевод строки");

lines.Add($"{t.Id};{t.Item};{t.Owner};{t.Due.ToString("dd.MM.yyyy",
CultureInfo.InvariantCulture)};{t.Status}");
        }
        File.WriteAllLines(path, lines, Encoding.UTF8);
    }

    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime
today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date <
today.Date);
}

internal static class OperatorView
{
    public static List<Ticket> Select(IEnumerable<Ticket> rows, DateTime
today) =>
        rows.Where(t => t.Status == "Открыта" && t.Due.Date < today.Date)
            .OrderBy(t => t.Due).ThenBy(t => t.Id).ToList();
}

public partial class MainWindow : Window
{
    private List<Ticket> tickets = new();
    private DateTime CheckDate => ControlDate.SelectedDate!.Value.Date;
```

1. MainWindow.xaml.cs — продолжение

```
    public MainWindow(string? demoPath = null, DateTime? checkDate = null)
    {
        InitializeComponent();
        ControlDate.SelectedDate = checkDate ?? new DateTime(2026, 10, 1);
        if (demoPath != null) Loaded += (_, _) =>
            Dispatcher.BeginInvoke(System.Windows.Threading.DispatcherPriority
.ApplicationIdle,
                new Action(() => LoadCsv(demoPath)));
    }

    private void ControlDate_Changed(object sender,
System.Windows.Controls.SelectionChangedEventArgs e)
    {
        if (ControlDate.SelectedDate is null)
            ControlDate.SelectedDate = new DateTime(2026, 10, 1);
    }

    private void LoadTestData_Click(object sender, RoutedEventArgs e) =>
LoadTestData();

    private void LoadTestData()
    {
        tickets = TestData.Create(CheckDate);
        TicketsGrid.ItemsSource = tickets.ToList();
        StatusText.Text = $"Тестовых записей: {tickets.Count}";
    }

    private void OpenCsv_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "CSV (*.csv)|*.csv" };
        if (dialog.ShowDialog() == true) LoadCsv(dialog.FileName);
    }
```

1. MainWindow.xaml.cs — продолжение

```
    private void LoadCsv(string path)
    {
        try
        {
            var result = FlexibleHeader.IsChecked == true ?
HeaderExchange.Read(path) : CsvStore.Read(path);
            tickets = result.Tickets;
            TicketsGrid.ItemsSource = tickets.ToList();
            StatusText.Text = $"Загружено: {tickets.Count}; пропущено:
{result.Warnings.Count}";
            if (result.Warnings.Count > 0)
                MessageBox.Show(string.Join(Environment.NewLine,
result.Warnings), "Предупреждения импорта");
        }
        catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка импорта");
}
    }

    private void SaveCsv_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new SaveFileDialog { Filter = "CSV (*.csv)|*.csv",
FileName = "export.csv" };
        if (dialog.ShowDialog() != true) return;
        try { CsvStore.Write(dialog.FileName, tickets); StatusText.Text =
"Файл сохранён"; }
        catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка экспорта");
 }
    }

    private void SelectForOperator_Click(object sender, RoutedEventArgs e)
    {
```

1. MainWindow.xaml.cs — продолжение

```
        var selected = OperatorView.Select(tickets, CheckDate);
        TicketsGrid.ItemsSource = selected;
        StatusText.Text = $"Для оператора: {selected.Count}; всего:
{tickets.Count}";
    }

    private void ShowAll_Click(object sender, RoutedEventArgs e)
    {
        TicketsGrid.ItemsSource = tickets.ToList();
        StatusText.Text = $"Все записи: {tickets.Count}";
    }

    private void CountOverdue_Click(object sender, RoutedEventArgs e) =>
        StatusText.Text = $"Просрочено: {CsvStore.CountOverdue(tickets,
CheckDate)}";
}
```

2. Проверка связей

Список, встроенные записи, выбранная дата и обработчики образуют один согласованный файл.

4.11. Расширение: полный файл HeaderExchange.cs для сверки

1. HeaderExchange.cs

```
using System.IO;
using System.Text;

namespace ServiceDeskWpf;

// Optional extension: adapt the order, then reuse the accepted strict reader.
internal static class HeaderExchange
{
    private static readonly string[] Columns = ["Id", "Item", "Owner", "Due",
"Status"];

    public static ReadResult Read(string path)
    {
        var lines = File.ReadAllLines(path, Encoding.UTF8);
        if (lines.Length == 0) throw new InvalidDataException("Отсутствует
заголовок CSV");
        var header = lines[0].TrimStart('\uFEFF').Split(';');
        if (header.Length != Columns.Length ||
header.Distinct(StringComparer.Ordinal).Count() != Columns.Length ||
            Columns.Any(name => !header.Contains(name,
StringComparer.Ordinal)))
            throw new InvalidDataException("Заголовок должен содержать Id,
Item, Owner, Due, Status по одному разу");
        var order = Columns.Select(name => Array.IndexOf(header,
name)).ToArray();
        var normalized = new List<string> { string.Join(';', Columns) };
        foreach (var line in lines.Skip(1))
        {
            if (string.IsNullOrWhiteSpace(line)) { normalized.Add(line);
continue; }
            var cells = line.Split(';');
            // Preserve malformed rows and physical line numbers for the
```

1. HeaderExchange.cs — продолжение

```
existing validator.
            normalized.Add(cells.Length == Columns.Length ? string.Join(';',
order.Select(i => cells[i])) : line);
        }
        var temp = Path.GetTempFileName();
        try { File.WriteAllLines(temp, normalized, Encoding.UTF8); return
CsvStore.Read(temp); }
        finally { File.Delete(temp); }
    }
}
```

2. Проверка связей

Адаптер добавляется только при выполнении расширенного маршрута. CsvStore.Read и CsvStore.Write основного контракта сохраняются.